

Construye e implementa un procedimiento en lenguaje ensamblador (x86 - 16bits) para escribir un byte a un puerto. El procedimiento debe programarse en lenguaje C considerando la siguiente función prototipo:

```
void outportb( uint16_t puerto, uint8_t dato );
```

Respuesta:

```
PROC  
; comentarios  
; protocolo de entrada a función  
; copiar parámetro dato  
; copiar parámetro puerto  
; sacar dato por puerto  
; protocolo de salida de función  
ret  
ENDP
```

```
outportb PROC  
; Escribir un byte en un puerto (x86 - 16 bits)  
; Parámetros:  
; puerto (uint16_t) - segmento [SP+4]  
; dato (uint8_t) - segmento [SP+6]  
  
push bp ; protocolo de entrada a función  
mov bp, sp  
  
mov al, [bp+6] ; copiar parámetro dato  
mov dx, [bp+4] ; copiar parámetro puerto  
out dx, al ; sacar dato por puerto  
  
pop bp ; protocolo de salida de función  
ret  
outportb ENDP
```

Complete la secuencia para escribir el valor de un **palabra** apuntada por **EDI** a la **dirección de puerto** contenida en **ESI**.

MOV ; tomar el dato

MOV ; tomar la dirección del puerto

; escribir dato al puerto

```
mov ax, [edi]
mov dx, esi
out dx, ax
```

Implemente un procedimiento en lenguaje ensamblador del 8088 (para después ser llamado desde lenguaje C) para activar un bit de un determinado dato el cual será pasado como parámetro por referencia, así como el número del bit a operar.

Time left

Considere:

```
typedef unsigned char uint8_t;
```

```
/*
```

```
* Prototipo:
```

```
* void SetBit ( uint8_t *dato, uint8_t nbit );
```

```
*/
```

```
* Ejemplo de uso:
```

```
* Si existe uint8_t valor=0x00
```

```
* active el bit 7 de valor
```

```
*/
```

```
SetBit( &valor, 7 ); /* Resultado: valor = 0x80 */
```

Respuesta:

```

PROC                                ; comentarios
; protocolo de entrada a función
; copiar parámetro nbit en CL
; copiar el otro parámetro en BX
mov al, 1                           ; máscara en AL
shl al, cl                          ; ajustar máscara
or byte ptr [bx], al                ; aplicar máscara a dato
; protocolo de salida a función
ENDP

```

SetBit PROC

; procedimiento que activa un bit

push bx

; protocolo de entrada a función

push cx

mov cl, byte ptr [sp+6] ; copiar parámetro nbit en CL

mov bx, word ptr [sp+4] ; copiar dirección del dato en BX

mov al, 1

; máscara en AL

shl al, cl

; ajustar máscara (1 << nbit)

or byte ptr [bx], al

; aplicar máscara al dato (activando el bit)

pop cx

; protocolo de salida a función

pop bx

ret

SetBit ENDP



* Ejemplo de uso:

* Si existe uint8_t valor=0x00

* activa el bit 7 de valor

*/

SetBit(&valor, 7); /* Resultado: valor = 0x80 */

Respuesta:

```
[ ] PROC                                ; comentarios
[ ]                                     ; protocolo de entrada a función
[ ]
[ ]                                     ; copiar parámetro nbit en CL
[ ]                                     ; copiar el otro parámetro en BX
mov al, [ ]                            ; mascara en AL
[ ] al, cl                             ; ajustar mascara
[ ]                                     ; aplicar mascara a dato
[ ]                                     ; protocolo de salida a función
[ ]
ENDP
```

Haciendo uso del procedimiento **putchar** escriba un procedimiento llamado **print_MSB** para desplegar en formato binario el bit (mas significativo) que se encuentra en el registro AX.

Forma de uso:

```
mov ax,873Fh ; el bit más significativo es 1 por tanto
call print_MSB ; se imprime carácter 1 ('1' ASCII)
```

Nota: El procedimiento **putchar** presenta en pantalla el caracter ASCII dado por el valor de AL
Por ejemplo: Si AL=41h el procedimiento **putchar** presenta 'A'

Procedimiento

```
print_MSB:
; procedimiento
; comentarios
; salva contenido de regs. a usar
mov eax,ax
; enmascarar bit de interés
and eax,1
; es cero
jz @@cero
; es uno, requiere ajuste
add al,1
@@cero: add al,0
; ajuste simple a ASCII para usar putchar
call putchar
pop eax
ret
; fin de procedimiento
```

Time left 0:32:33

* Ejemplo de uso:

* Si existe uint8_t valor=0x00

* activa el bit 7 de valor

*/

SetBit(&valor, 7); /* Resultado: valor = 0x80 */

Respuesta:

```
[ ] PROC                                     ; comentarios
    [ ]                                     ; protocolo de entrada a función
    [ ]
    [ ]                                     ; copiar parámetro nbit en CL
    [ ]                                     ; copiar el otro parámetro en BX
    mov al, [ ]                             ; mascara en AL
    [ ] al, cl                             ; ajustar mascara
    [ ]                                     ; aplicar mascara a dato
    [ ]                                     ; protocolo de salida a función
    [ ]
    ENDP
```

Implemente un procedimiento en lenguaje ensamblador del 8088 (para después ser llamada desde lenguaje C) para activar un bit de un determinado dato el cual será pasado como parámetro por referencia, así como el número del bit a operar.

Time left

Considere:

```
typedef unsigned char uint8_t;
```

```
/*
```

```
* Prototipo:
```

```
* void SetBit ( uint8_t *dato, uint8_t nbit );
```

```
*
```

```
* Ejemplo de uso:
```

```
* Si existe uint8_t valor=0x00
```

```
* activa el bit 7 de valor
```

```
*/
```

```
SetBit( &valor, 7 ); /* Resultado: valor = 0x80 */
```

Respuesta:

PROC

; comentarios

; protocolo de entrada a función

; copiar parámetro nbit en CL

; copiar el otro parámetro en BX

mov al,

; mascara en AL

al,cl

; ajustar mascara

; aplicar mascara a dato

Diseñe e implemente un procedimiento en **lenguaje ensamblador** (x86 -16bits) para escribir un **byte** a un puerto. El procedimiento debe ser llamado desde programas en **lenguaje C** considerando la siguiente función prototipo:

Time left 0:33:09

Respuesta:

```
void outportb( uint16_t puerto, uint8_t dato );
```

PROC

; comentarios

; protocolo de entrada a función

; copiar parámetro dato

; copiar parámetro puerto

; sacar dato por puerto

; protocolo de salida de función

ret

ENDP

Time left

¿Cuántos chips de memoria RAM de 128x4 se necesitan para proveer una capacidad de memoria de 16,384 bytes?, ¿cuántos bits tendrá cada dirección del chip?

Select one:

- ☐ 256 chips de memoria. Cada dirección tendría 4 bits.
- ☐ 256 chips de memoria. Cada dirección tendría 14 bits.
- ☐ 128 chips de memoria. Cada dirección tendría 14 bits.
- ☐ Es necesario mas información.

AX=0075h, BX=0CB0h, CX=EF01h, DX=807Fh, BP=0400h, SP=8A24h.

CMP DL,DH

JA <LABEL> ; ¿existe salto?

CMP AH,BH

JL <LABEL> ; ¿existe salto?

CMP BL,DL

JB <LABEL> ; ¿existe salto?

CMP BX,AX

JMP AX ; ¿existe salto?

ROL CX,1

JC <LABEL> ; ¿existe salto?

TEST BX,1FH

JZ <LABEL> ; ¿existe salto?

RCR AX,1

TEST AX,2

JNC <LABEL> ;

INC AX

JP <LABEL> ;

No

Sí

No

Sí

Sí

No

No

No

Seleccione la opción correcta para indicar si existe salto o no según las instrucciones y valores de registro
AX=D075h, BX=0CB0h, CX=EF01h, DX=807Fh, BP=0400h, SP=8A24h.

CMP DL,DH

JA <LABEL> ; ¿existe salto?

CMP AH,BH

JL <LABEL> ; ¿existe salto?

Si

No

Se requiere mas información

CMP BL,DL

JB <LABEL> ; ¿existe salto?

CMP BX,AX

JMP AX ; ¿existe salto?

ROL CX,1

JC <LABEL> ; ¿existe salto?

TEST BX,1FH

JZ <LABEL> ; ¿existe salto?

RCR AX,1

TEST AX,2

JNC <LABEL> ;

No

Sí

No

Sí

Sí

No

No

>

SI

Seleccione el modo de direccionamiento correcto:

MOV AX,DS; de registro  

MOV AL,[NUM]; directo  

MOV [LIST],[BX]; no permitido  

MOV AX,[BX-10]; relativo a registro  

MOV BX,AL; no permitido  

MOV [EBX+ESI*2],EAX; relativo a base+indice  

MOV DS,ES; no permitido  

MOV ECX,0CH; inmediato  

Escriba un procedimiento en asm que convierta un número **BCD** de máximo 2 dígitos pasado como parámetro en AL su valor (**numérico**), el valor resultante es retornado en AL.

Por ejemplo:

Si la entrada es **AL=14h** (14 BCD) entonces el procedimiento, retorna **AL=0Eh** o bien AL=14 decimal

Si la entrada es **AL=10h** (10 BCD) entonces el procedimiento, retorna **AL=0Ah** o bien AL=10 decimal

Si la entrada es **AL=99h** (99 BCD) entonces el procedimiento, retorna **AL=63h** o bien AL=99 decimal

BCD_TO_BIN:

```
push bx      ; Guarda BX
push cx      ; Guarda CX
```

```
mov bl, al   ; BL = DU (BCD original)
and al, 0Fh  ; AL = U (Unidades, se guardan para la suma final)
```

```
and bl, F0h  ; BL = D0 (Decenas * 16)
mov cl, 4    ; CL = 4
shr bl, cl   ; BL = D (Decenas, valor numérico)
```

```
; Multiplicación de Decenas (D * 10)
```

```
mov cl, 10   ; CL = 10
xchg al, bl  ; AL = D (Decenas), BL = U (Unidades)
mul cl       ; AX = AL (D) * 10. (El resultado de la multiplicación por 10 queda en
AL)
```

```
; Suma final
add al, bl   ; AL = (D * 10) + U. (BL contiene las Unidades 'U')
```

```
pop cx       ; Restaura CX
pop bx       ; Restaura BX
ret          ; Resultado en AL
```

Question 18

Not yet
answered

Marked out of
8.0

 Remove flag

Considera el siguiente contenido de registros y de memoria:

Registros:

EAX=10000535h, EBX=0001000Fh, ECX=100FFF45h, EDX=012302F0h, ESI=0000000Ch, EDI=00004701h, EBP=00000008h, ESP=010100FF

Time left 0:08:3

Memoria:

(inicio de segmento en 0)

Dirección	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0000	53	54	50	00	46	49	53	54	-46	4D	55	4C	55	4C	00	46
0010	53	54	00	48	44	04	56	00	-50	00	46	44	46	44	49	56
0020	00	49	4E	43	4F	00	49	4E	-49	56	00	46	52	00	46	49
0030	00	49	52	45	42	45	00	4A	-46	57	41	49	4C	44	00	46
0040	46	53	54	00	50	00	46	49	-00	46	49	4D	44	49	56	52
0050	4C	54	00	49	49	4D	55	4C	-49	56	52	00	50	00	46	44
0060	00	49	4E	54	54	00	49	4E	-49	44	49	56	44	49	56	00
0070	54	00	4A	4E	41	45	00	4A	-54	00	46	49	4C	44	00	46

Determine el valor del registro afectado según la ejecución de la instrucción:

a) INC CH ; Registro ECX= 100F0045 h

b) ADD BL,[ESI-2] ; Registro EBX= 0001006 h

c) SUB DI, [BX+2] ; Registro EDI= 000046AD h

d) DEC word ptr [SI+4] ; Memoria: word ptr [SI+4]= 544F h

e) MUL DL ; Registro afectado= EAX valor= 100031B0 h

Question 13

Incomplete
answer

Marked out of
10.0

 Remove flag

Please answer all parts of the question.

Considera el siguiente contenido de registros y de memoria:

Registros:

AX=1234h, BX=ABCDh, CX=4502h, DX=0FF0h, DS=09A0h, SS=09A2h, BP=0008h, SI=0002h, DI=0004h
FLAGS=0026h;

Memoria:

Dirección	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0000	53	54	50	00	46	49	53	54	-46	4D	55	4C	55	4C	00	46
0010	53	54	00	48	44	04	56	00	-50	00	46	44	46	44	49	56
0020	00	49	4E	43	4F	00	49	4E	-49	56	00	46	52	00	46	49
0030	00	49	52	45	42	45	00	4A	-46	57	41	49	4C	44	00	46
0040	46	53	54	00	50	00	46	49	-00	46	49	4D	44	49	56	52
0050	4C	54	00	49	49	4D	55	4C	-49	56	52	00	50	00	46	44
0060	00	49	4E	54	54	00	49	4E	-49	44	49	56	44	49	56	00
0070	54	00	4A	4E	41	45	00	4A	-54	00	46	49	4C	44	00	46

Determine el valor del registro afectado según la ejecución de la instrucción, ignorando el registro de segmento:

a) AND BL, [BP-6] ; BX= H

b) OR AX,[SI+BP-1] ; AX= H

c) XOR DL,[SI+6] ; DX= H

d) SHR AX,CL ; AX= H

d) RCL CL,1 ; CL= H

e) NEG CL ; CL= H

f) CBW ; Registro afectado= , valor= H

g) NOT word ptr [SI+1] ; Lugar de memoria [SI+1] = H

Time left 0:0

Question 14

Not yet
answered

Marked out of
10.0

 Remove flag

Haciendo uso del procedimiento **putchar** escriba un procedimiento llamado **print_MSB** para desplegar en formato binario el **bit** (mas significativo) que se encuentra en el registro AX.

Time left 0:11:21

Forma de uso:

```
mov ax,873Fh ; el bit más significativo es 1 por tanto
call print_MSB ; se imprime carácter 1 ('1' ASCII)
```

Nota: El procedimiento **putchar** presenta en pantalla el caracter ASCII dado por el valor de AL
Por ejemplo: Si AL=41h el procedimiento **putchar** presenta 'A'

Procedimiento

```
print_MSB:      ; procedimiento      ; comentarios
                push cx              ; salva contenido de regs. a usar
                and ax, 8000h        ; enmascarar bit de interés
                je @@cero            ; es cero
                add al, 01h          ; es uno, requiere ajuste
@@cero: add al, 00h                  ; ajuste simple a ASCII para usar putchar
                call putchar
                pop eax
                ret
                ; fin de procedimiento
```

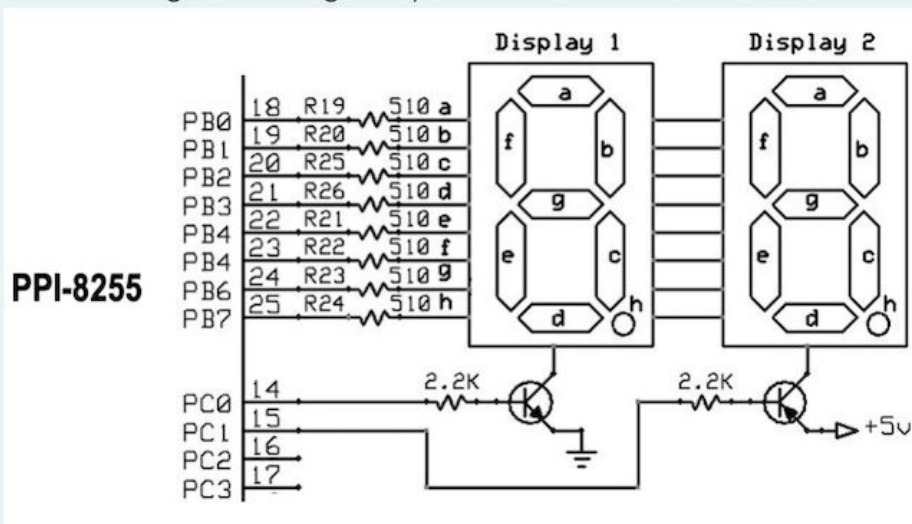
Question 19

Incomplete
answer

Marked out of
4.0

 Remove flag

En base al siguiente diagrama, determine los valores de la terminales acorde a la que se pide.



Determine el **numero (hex)** desplegado en cada display si **PB=86h** y **PCL=Ah**.

Display1 presenta:

Display2 presenta:

Question 23

Answer saved

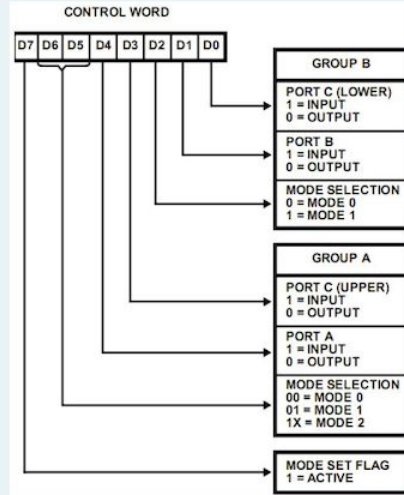
Marked out of 5.0

[Remove flag](#)

Dado un **PPI-8255** con dirección base **4FH**, escriba un procedimiento para programar en modo 0 el puerto C de salida, puerto B y puerto A de entrada.

Prog_8255 **PROC** ; procedimiento

```
push DX ✓  
push AX ✓  
MOV DX, 52h ✗  
mov AX, 92 ✓ h  
out DX ✓, AL ✓  
pop AX ✓  
pop DX  
ret  
ENDP ; -----
```



Question 22

Not yet
answered

Marked out of
6.0

 Remove flag

Diseñe e implemente un procedimiento llamado **printBin16b** el cual imprime en pantalla el valor de **AX** en formato binario. Por ejemplo si **AX=1234h** entonces se imprime en pantalla **0001001000110100**

Notas:

1. Haga uso del procedimiento **print_MSB** -- ejercicio anterior
2. Puede hacer uso del procedimiento **putchar** el cual presenta en pantalla el caracter ASCII dado por el valor de AL
Por ejemplo: Si AL='X' el procedimiento **putchar** presenta **X** en pantalla.

printBin16b:

```
push ax      ; Guarda el valor original de AX, aunque print_MSB lo restaura, es buena práctica.
push cx      ; Guarda el contador de bucle
```

```
mov cx, 16   ; Inicializa el contador para 16 bits
```

@loop_bit:

```
; 1. Imprime el MSB (bit 15) del valor actual de AX
call print_MSB ; Imprime '0' o '1' basado en AX[15]
```

```
; 2. Prepara el siguiente bit
```

```
rol ax, 1    ; Rota AX a la izquierda por 1 posición. El siguiente bit se mueve a AX[15].
```

```
loop @loop_bit ; Decrementa CX y salta a @loop_bit si CX != 0
```

```
pop cx       ; Restaura CX
pop ax       ; Restaura el AX original (el valor que se imprimió)
ret
```

Question 20

Not yet
answered

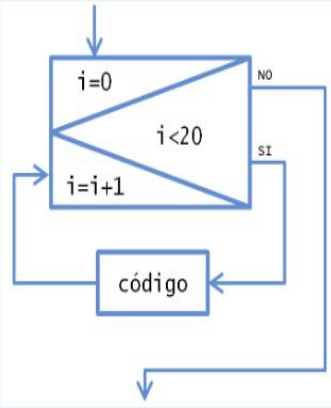
Marked out of
8.0

🚩 Flag
question

Llenar los espacios correctamente acorde a la estructura de alto nivel que se muestra donde i es una variable entera ya definida:

Diagrama de flujo

código (ASM)



```

:
mov [i],0 ✓
@@label2: cmp [i],20 ✓
@@label1: jae @@label3 ✓
; - - código a ciclar - - ✓
inc [i] ✓
jmp @@label2 ✓
@@label3: :
```

@@label1:

loop

ja

inc byte ptr[i]

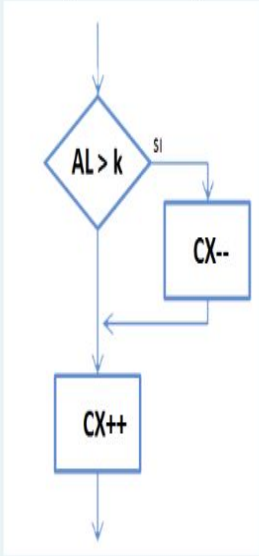
cmp byte ptr[i]

jbe

@@label2:

Llenar los espacios correctamente acorde a la estructura de alto nivel que se muestra y acorde a que **k** es un entero de 8 bits:

Diagrama de flujo



código (ASM)

```
      :  
      cmp al,k  
      jle @@label2  
@@label1: dec cx  
@@label2: inc cx  
      :
```

@@label1	ja	jg	jge	jbe	@@label2	cmp [k],al	cmp al,[k]
----------	----	----	-----	-----	----------	------------	------------

Escriba la secuencia para que dado un valor en AL=XXXXXXXX se llegue AL=1X0X0X1X
Donde: X significa bit negado.

Secuencia de instrucciones en lenguaje ensamblador:

; instrucción para poner unos

; instrucción para invertir

; instrucción para poner ceros

OR AL, 82h
XOR AL, 55h
AND AL, AFh

¿Para qué se utilizan los registros de propósito general?

Select one:

- ☐ a. Para que el programador los use como quiera.
- ☒ b. Para operaciones generales y algunas específicas según el registro usado. ✓
- ☐ c. Para almacenar solamente datos de forma temporal.
- ☐ d. Para operaciones generales solamente.

¿Para qué se utilizan los registros apuntadores y de índice?

Select one:

- ☐ a. Para que el programador los use como el quiera.
- ☐ b. Para trabajar con segmentos de memoria.
- ☒ c. Para indicar o apuntar una localidad de memoria dentro del un segmento de memoria. ✓
- ☐ d. Para operaciones generales solamente con direcciones de memoria.

¿Un **procesador** para qué utiliza el registro de banderas?

Select one:

- ☐ a. Es un registro que contiene información sobre el estado de operaciones de la ALU.
- ☒ b. Para indicar información sobre el estado de operaciones de la ALU. ✓
- ☐ c. Para tomar decisiones en base a la información contenida en este registro.
- ☐ d. No se puede acceder.

¿Para qué se utilizan los registros de segmento?

Select one:

- ☐ a. Para contener código, datos y pila.
- ☐ b. Para almacenar solamente datos de forma temporal.
- ☒ c. Para colaborar en el direccionamiento a zonas de memoria. ✓
- ☐ d. No se pueden usar.

Seleccione el texto correcto según el tipo de registro y tamaño en bits.

BX :	Registro de Proposito General	✓	tamaño:	16	✓
DI :	Registro Apuntador e Indice	✓	tamaño:	16	✓
EDI :	Registro Apuntador e Indice	✓	tamaño:	32	✓
SS :	Registro de Segmento	✓	tamaño:	16	✓
DX :	Registro de Proposito General	✓	tamaño:	16	✓
SI :	Registro Apuntador e Indice	✓	tamaño:	16	✓
ECX :	Registro de Proposito General	✓	tamaño:	32	✓
AX :	Registro de Proposito General	✓	tamaño:	16	✓
DS :	Registro de Segmento	✓	tamaño:	16	✓
EBX :	Registro de Proposito General	✓	tamaño:	32	✓
BP :	Registro Apuntador e Indice	✓	tamaño:	16	✓
CX :	Registro de Proposito General	✓	tamaño:	16	✓
FLAGS:	Registro de Banderas	✓	tamaño:	16	✓
SP :	Registro Apuntador e Indice	✓	tamaño:	16	✓
CS :	Registro de Segmento	✓	tamaño:	16	✓
ESI :	Registro Apuntador e Indice	✓	tamaño:	32	✓
IP :	Registro Apuntador e Indice	✓	tamaño:	16	✓
EDX :	Registro de Proposito General	✓	tamaño:	32	✓
ES :	Registro de Segmento	✓	tamaño:	16	✓
EBP :	Registro Apuntador e Indice	✓	tamaño:	32	✓
FS :	Registro de Segmento	✓	tamaño:	16	✓
ESP :	Registro Apuntador e Indice	✓	tamaño:	32	✓
GS :	Registro de Segmento	✓	tamaño:	16	✓
EAX :	Registro de Proposito General	✓	tamaño:	32	✓
EFLAGS:	Registro de Banderas	✓	tamaño:	32	✓
EAX :	Registro de Proposito General	✓	tamaño:	32	✓
EIP :	No existe	✗	tamaño:	32	✓

Expresa la dirección lógica a la que accede la instrucción:

Nota: Los términos de la expresión deberá según el orden: base, índice, desplazamiento numérico (con terminación h) y **sin espacios**; siempre y cuando la expresión contenga esos términos.

Si no se respecta ese orden la respuesta será errónea por el sistemas que evalúa la expresión.

Ejemplo:

Instrucción	Dirección lógica a la que accede
	Segmento: Offset
MOV BX, DIR[5522H]	DS : DIR+5522H

Instrucción	Dirección lógica a la que accede	
	Segmento : Offset	
MOV CX,[DI]	DS ✓	: DI ✓
MOV [100H],BP	DS ✓	: 100H ✓
MOV BX,[BX+100H]	DS ✓	: BX+100H ✓
MOV LIST[BP],BX	SS ✓	: BP+LIST ✓
MOV BP,[DI][10H]	DS ✓	: DI+10H ✓
MOV [BP+200H+SI],BP	SS ✓	: BP+SI+200H ✓
MOV BX,[1234H]	DS ✓	: 1234H ✓
MOV BH,[BP+SI]	SS ✓	: BP+SI ✓
MOVBX,DATOS[DI+120H]	DS ✓	: DI+120H+DATOS ✓
MOV [BP],BX	SS ✓	: BP ✓

Escriba el resultado según la instrucción, considerando cada instrucción por separado de las demás y lo siguiente:

AX=0040h, BX=0002h, CX=0C02h, DX=8208h, SI=0002h, DI=0048, BP=0008h, SP=42A8h, FLAGS=8065h

CS=001Bh, SS=09A1h, DS=09A0h, ES=0038h

Tambien existen 4 puertos con la siguiente informacion

- P1 con direccion 8205 y contiene el valor 0082h
- P2 con direccion 8206 y contiene el valor 0035h
- P3 con direccion 8207 y contiene el valor 0062h
- P4 con direccion 8208 y contiene el valor 0006h

Considera el siguiente contenido de memoria:

09A0:0000	46	4D	55	4C	00	46	49	4D-55	4C	00	46	44	49	56	52
09A0:0010	50	00	46	44	49	56	52	00-46	44	49	56	50	00	46	44
09A0:0020	49	56	00	46	49	44	49	56-52	00	46	49	44	49	56	00
09A0:0030	46	57	41	49	54	00	46	49-4C	44	00	46	4C	44	00	46
09A0:0040	53	54	50	00	46	53	54	00-46	49	53	54	50	00	46	49
09A0:0050	53	54	00	48	4C	54	00	49-44	49	56	00	49	4D	55	4C
09A0:0060	00	49	4E	43	00	49	4E	54-4F	00	49	4E	54	00	49	4E
09A0:0070	00	49	52	45	54	00	4A	4E-42	45	00	4A	41	45	00	4A

1. XCHG AH, CH ; AX= 0C40 ✓ h CX= 0002 ✓ h

2. XCHG AL,[BX+DI] ; AX= 0053 ✓ h BX= 0002 ✓ h DI= 0048 ✓ h

3. IN AX,P2 ; AX= 0035 ✓ h P2= 0035 ✓ h

4. OUT DX,AL ; AX= 0040 ✓ h DX= 8208 ✓ h P1= 0082 ✓ h P2= 0035 ✓ h P3= 0062 ✓ h P4= 0040 ✓ h

5. LAHF ; AX= 6540 ✓ h

6. XLAT ; AX= 0050 ✓ h

7. LDS AX,[DI] ; AX= 4946 ✓ h DS= 5453 ✓ h

8. MOVZX AX, CL ; AX= 0002 ✓ h CX= 0C02 ✓ h

Considerando los siguientes valores de registros:

EAX=1234567B, EBX=FFFF000F, ECX=00FF01FF, EDX=0A0B0123, EBP=00000023, ESI=FF00000A, EDI=1200FFFF

Determine el resultado y registro afectado

1. ADD AL,DH ; AL= 7C ✓ h
2. INC ECX ; ECX= 00FF0200 ✓ h
3. ADD BL,CH ; BL= 10 ✓ h
4. DEC DL ; DL= 22 ✓ h
5. DEC SI ; SI= 0009 ✓ h
6. INC EDI ; EDI= 12010000 ✓ h
7. SUB ECX,0FFh ; ECX= 00FF0100 ✓ h
8. CBW ; AX= 007B ✓ h
9. CWD ; DX= 0000 ✓ h , AX= 567B ✓ h
10. CWDE ; EAX= 0000567B ✓ h
11. MUL CL ; AX= 7A85 ✓ h
12. IMUL CL ; AX= FF85 ✓ h
13. MUL BX ; DX= 0005 ✓ h , AX= 1135 ✓ h
14. DIV CL ; AL= 56 ✓ h , AH= D1 ✓ h
15. DIV DI ; AX= 0123 ✓ h , DX= 579E ✓ h

Considerando los siguientes valores de registros:

EAX=1234567B, EBX=FFFF000F, ECX=00FF01FF, EDX=0A0B0123, EBP=00000023, ESI=FF00000A, EDI=1200FFFF

Determine el resultado y registro afectado

1. AND AL, 0 ; AL= 00 ✓ h
2. AND DL, 0Fh ; DL= 03 ✓ h
3. AND DX, 0F0F ; DX= 0103 ✓ h
4. OR BP, 1F00h ; BP= 1F23 ✓ h
5. OR EDI,120000FF ; EDI= 1200FFFF ✓ h
6. NOT AH ; AH= A9 ✓ h
7. NOT CX ; CX= FE00 ✓ h
8. XOR AL,81h ; AL= FA ✓ h
9. XOR DX,FFh ; DX= 01DC ✓ h
10. XOR BP,BP ; BP= 0000 ✓ h

Escriba la secuencia para que dado un valor en **AL=XXXX-XXXXH** se llegue **AL=0XX1-0XX1H**

Nota: El bit subrayado significa bit negado.

Secuencia en lenguaje ensamblador:

AND AL, 77H



; instrucción para poner
ceros

OR AL, 11H



; instrucción para poner
unos

XOR AL, 24H



; instrucción para invertir

Escriba el resultado según la instrucción considerando:

AX=57D0h, BX=C00Bh, CX=3421h, BP=0008h, SP=42A8h

a) SAR BX, 1 ; BX= E005



h

b) ROR BX, CL ; BX= E005



h

c) SHL BX, 1 ; BX= 8016



h

d) NEG AL ; AL= 30



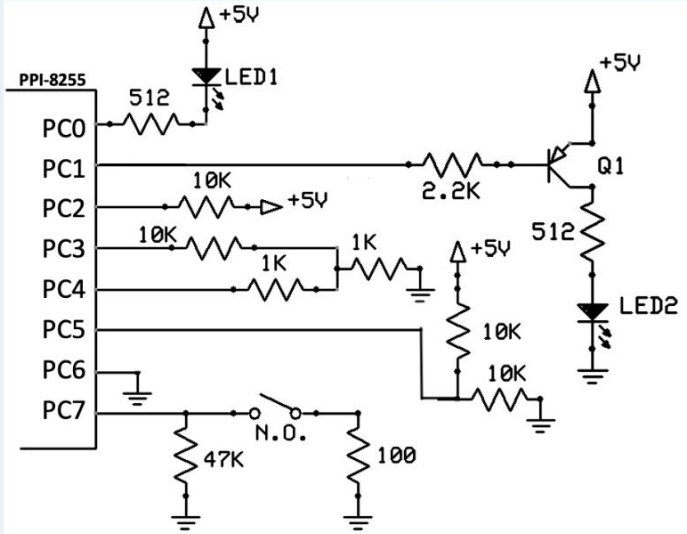
h

e) CBW ; AX= FFD0



h

Considerando la figura y que el **PCL** está configurado de **salida** y **PCH** de **entrada**, indique el valor (lógico) de las terminales correspondiente a una lectura llenando cada posición del bit correspondiente asumiendo que **primero se le escribió** el dato **FAh** al puerto C.



Los posibles valores son **0** (cero lógico), **1** (1 lógico) y **U** (desconocido).

Terminal	PC7	PC6	PC5	PC4	PC3	PC2	PC1	PC0
Valor lógico	U	1	1	1	1	0	1	0
	✗	✗	✗	✗	✓	✓	✓	✓

Considerando el circuito de la figura indique el estado de los LEDs (ON o OFF).

LED1: ON ✓ LED2: OFF ✓

Se desea hacer la operación **AND** entre el registro AX y BX con resultado en AX; Escriba la instrucción para ello:

AND AX, BX

 ✓

Ahora considerando la inexistencia de la instrucción **AND** escriba una secuencia de 4 instrucciones para realizar la misma operación.

instrucción	comentario
NOT AX	; invertir operando
NOT BX	✓
OR AX, BX	✓
NOT AX	; invertir resultado

Escriba el resultado según la instrucción considerando:

AX=57D0h, BX=C00Bh, CX=3404h, DX=1234h BP=0008h, SP=42A8h, FLAGS=7203h

- 1. SHL CH, 1 ; CH= 68 ✓ h
- 2. SHR BX, CL ; BX= 0C00 ✓ h
- 3. SAL CH,1 ; CH= 68 ✓ h
- 4. SAR BX, 1 ; BX= E005 ✓ h
- 5. ROR AL,1 ; AL= 68 ✓ h
- 6. ROR DX,CL ; DX= 4123 ✓ h
- 7. ROL DL,1 ; DL= 68 ✓ h
- 8. ROL BX,CL ; BX= 00BC ✓ h
- 9. RCL BL,1 ; BL= 17 ✓ h
- 10. RCR BL,1 ; BL= 85 ✓ h

Considera el siguiente contenido de memoria:

09A0:0000	46	53	54	00	50	00	46	49	-	53	54	50	00	46	49	53	54
09A0:0010	4C	54	00	49	49	4D	55	4C	-	53	54	00	48	44	49	56	00
09A0:0020	00	49	4E	54	54	00	49	4E	-	00	49	4E	43	4F	00	49	4E
09A0:0030	54	00	4A	4E	41	45	00	4A	-	00	49	52	45	42	45	00	4A
09A0:0040	00	46	49	4D	44	49	56	52	-	46	4D	55	4C	55	4C	00	46
09A0:0050	49	56	52	00	50	00	46	44	-	50	00	46	44	46	44	49	56
09A0:0060	49	44	49	56	44	49	56	00	-	49	56	00	46	52	00	46	49
09A0:0070	54	00	46	49	4C	44	00	46	-	46	57	41	49	4C	44	00	46

Determine el valor de los registros afectados

a) LDS DI,[BX] ; Si BX=0000H, DS=09A0H, ES=09A1H y DI=0020H después de ejecutar se tiene DI= 5346 ✓ H y DS= 0054 ✓ H

b) LES SI,[DI] ; Si SI=0000H, DS=09A1H, ES=09A2H y DI=0020H después de ejecutar se tiene SI= 0054 ✓ H y ES= 4E4A ✓ H

Seleccione la opción correcta para indicar si existe salto o no según las instrucciones y valores de registros:

AX=1234h, BX=FF01h, CX=EF02h, DX=7F80h, BP=0004h, SP=248Ah.

CMP AL,DL

JA <LABEL> ; ¿existe salto?

No



CMP AH,BH

JL <LABEL> ; ¿existe salto?

No



CMP BL,DL

JB <LABEL> ; ¿existe salto?

Si



CMP BX,AX

JMP AX ; ¿existe salto?

Si



ROL CX,1

JC <LABEL> ; ¿existe salto?

Si



TEST BX,1FH

JZ <LABEL> ; ¿existe salto?

No



ADD AX,1

JC <LABEL> ; ¿existe salto?

No



Determine el estado final de la memoria, una vez que se ha ejecutado la siguiente secuencia de instrucciones según las condiciones

Time left 0

Estado inicial según definición:

section .data

NUMS db 30h, 31h, 32h, 33h, 34h, 35h, 36h, 37h, 38h, 39h ; '0123456789'

ALFAS db 41h, 42h, 43h, 44h, 45h, 46h, 47h, 48h, 49h, 4Ah ; 'ABCDEFGHIJ'

Secuencia:

mov esi, 4

mov ebx, NUMS

mov ah, [ebx+4]

mov al, [ebx+esi*4 - 5 + 3]

mov [ebx+esi*2+2], ah

mov [ebx+1], al

Estado final de la section .data :

NUMS: h, h, h, h, h, h, h, h, h, h

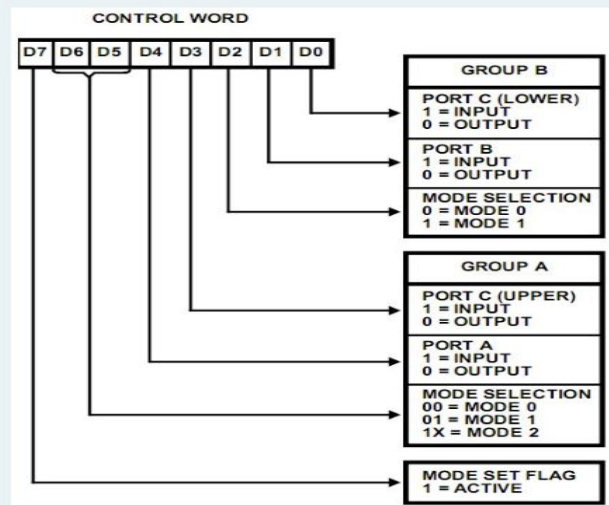
ALFAS: h, h, h, h, h, h, h, h, h, h

- **NUMS:** 30h, 45h, 32h, 33h, 34h, 35h, 36h, 37h, 38h, 39h
- **ALFAS:** 34h, 42h, 43h, 44h, 45h, 46h, 47h, 48h, 49h, 4Ah

Dado un **PPI-8255** con dirección base **40H**, determine la palabra de control según la configuración requerida en **modo 0**.

Puerto A Puerto B Puerto C (alto) Puerto C (bajo) Palabra de Control

IN	IN	IN	IN	9B ✓ h
OUT	OUT	OUT	OUT	80 ✓ h
OUT	IN	IN	IN	8B ✓ h
IN	OUT	IN	IN	99 ✓ h
IN	IN	OUT	OUT	92 ✓ h



Dado un **PPI-8255** con dirección base **4FH**, escriba un procedimiento para programar en modo 0 el puerto C de salida, puerto B y puerto A de entrada.

```
Prog_8255  PROC ; procedimiento

push DX ✓

push AX ✓

MOV AL, 90H ✗

mov AX, 52h ✗ h

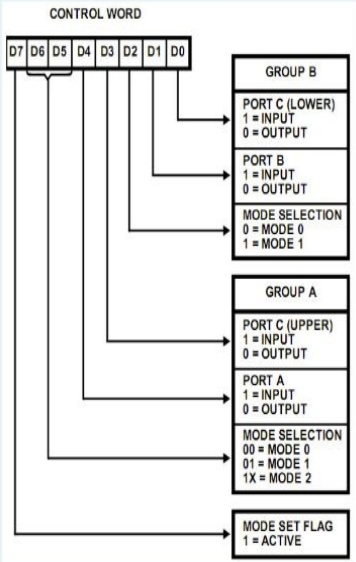
out DX ✓ , AL ✓

pop AX ✓

pop DX

ret

ENDP ; -----
```



Describe la diferencia entre **Organización de las Computadoras** y **Arquitectura de Computadoras**

Time left 0:10:22



- La arquitectura especifica **qué** debe hacer el sistema
- La organización define **cómo** se lleva a cabo

¿Defina las 4 funciones básicas de una **computadora**?



1. Procesamiento de datos
2. Almacenamiento de datos
3. Transferencia de datos
4. Control

Tomando como base las 4 funciones principales de un procesador defina que es un microprocesador.



Es una unidad central de procesamiento (CPU) contenida en un solo circuito integrado (chip).

Seleccione las características exclusivas que corresponden a la filosofía **CISC**:

Select one or more:

- a. Procesamiento serie de varias etapas
-  b. Instrucciones ejecutadas por microprograma
- c. Instrucciones sencillas en un ciclo
-  d. Cualquier instrucción puede referenciar a memoria
-  e. Muchas instrucciones y modos
- f. La complejidad está en el compilador
- g. Varios conjuntos de registros

“Una técnica para mejorar el rendimiento en la que el procesador mira hacia adelante en el código de instrucción extraído de la memoria y predice qué bifurcaciones, o grupos de instrucciones, es probable que se procesen a continuación.”

Las opciones son:

- a. Pipelining
-  b. Predicción de salto
- c. Ejecución superescalar
- d. Ejecución especulativa

Seleccione las características exclusivas que corresponden a la filosofía CISC

Select one or more:

- ☐ a. Procesamiento serie de varias etapas
- ☒ b. Instrucciones ejecutadas por microprograma
- ☐ c. Instrucciones sencillas en un ciclo
- ☒ d. Cualquier instrucción puede referenciar a memoria
- ☒ e. Muchas instrucciones y modos
- ☐ f. La complejidad está en el compilador
- ☐ g. Varios conjuntos de registros

¿Cuántos chips de memoria RAM de 128×4 se necesitan para proveer una capacidad de memoria de 16,384 bytes?, ¿dirección del chip?

Select one:

- ☐ 256 chips de memoria. Cada dirección tendría 4 bits.
- ☒ 256 chips de memoria. Cada dirección tendría 14 bits.
- ☐ 128 chips de memoria. Cada dirección tendría 14 bits.
- ☐ Es necesario mas información.

En programas en lenguaje ensamblador x86 (32-bits) ¿Cuales registros se pueden usar en las instrucciones para direccionar a memoria?

Select one or more:

- AX
- BX
- CX
- DX
- SI
- DI
- BP
- SP
- IP
- FLAGS
- EAX ☒
- EBX ☒
- ECX ☒
- EDX ☒
- EBP ☒
- ESP ☒
- ESI ☒
- EDI ☒

En programas en lenguaje ensamblador x86 (16-bits) ¿Cuáles registros se pueden usar en las instrucciones para direccionar a memoria?

Select one or more:

AX

EX

CX

DX

SI

DI

BP

SP

IP

FLAGS

EAX

EBX

ECX

EDX

EBP

ESP

ESI

EDI

